TicketSenpai

A concert-ticketing application for a local web-security capstone

Milestone 1 packages a working ticket-purchase flow with six intentional security weaknesses for controlled peer testing. All events, customer accounts, payments, and tickets are synthetic. The application reuses the existing ticketing concept and visual language in the required full-stack architecture.

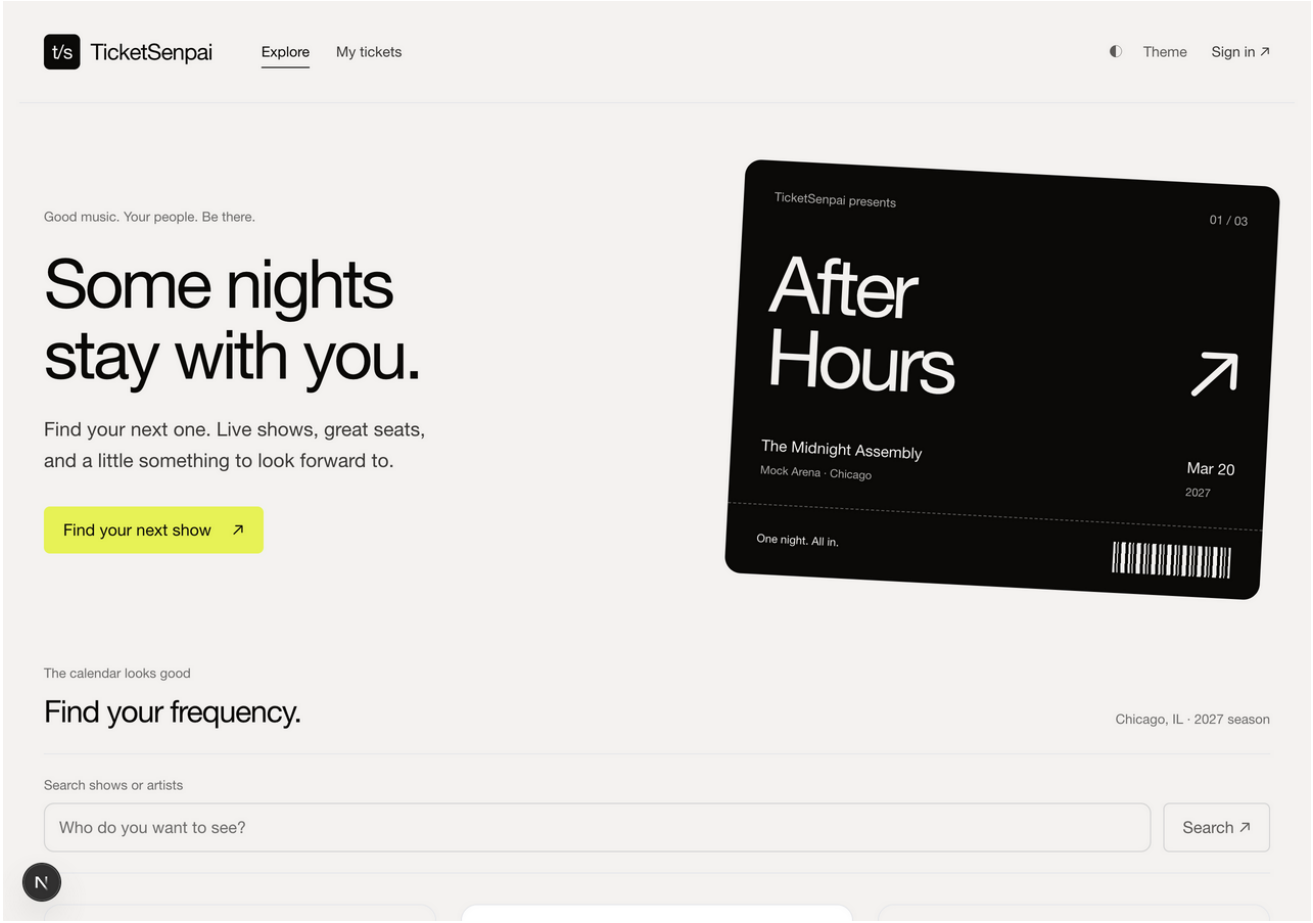


Figure 1. Event discovery at /events, captured from the running application.

Architecture and scope

Browser / Next.js 16 + React 19 on port 3000 → same-origin /api proxy → Flask 3.1 on port 5001 → SQLite. Both development servers bind to 127.0.0.1.

Five feature groups: accounts and profiles; event discovery and search; seat holds and simulated checkout; persisted event reviews; customer tickets and administrator event management. Three published fictional shows and one draft are seeded automatically.

State model: events own seats; users create holds, orders, reviews, and sessions. SQLite transactions and unique order-item constraints preserve inventory integrity while explicitly marked LAB V01-V06 paths remain vulnerable.

The customer journey

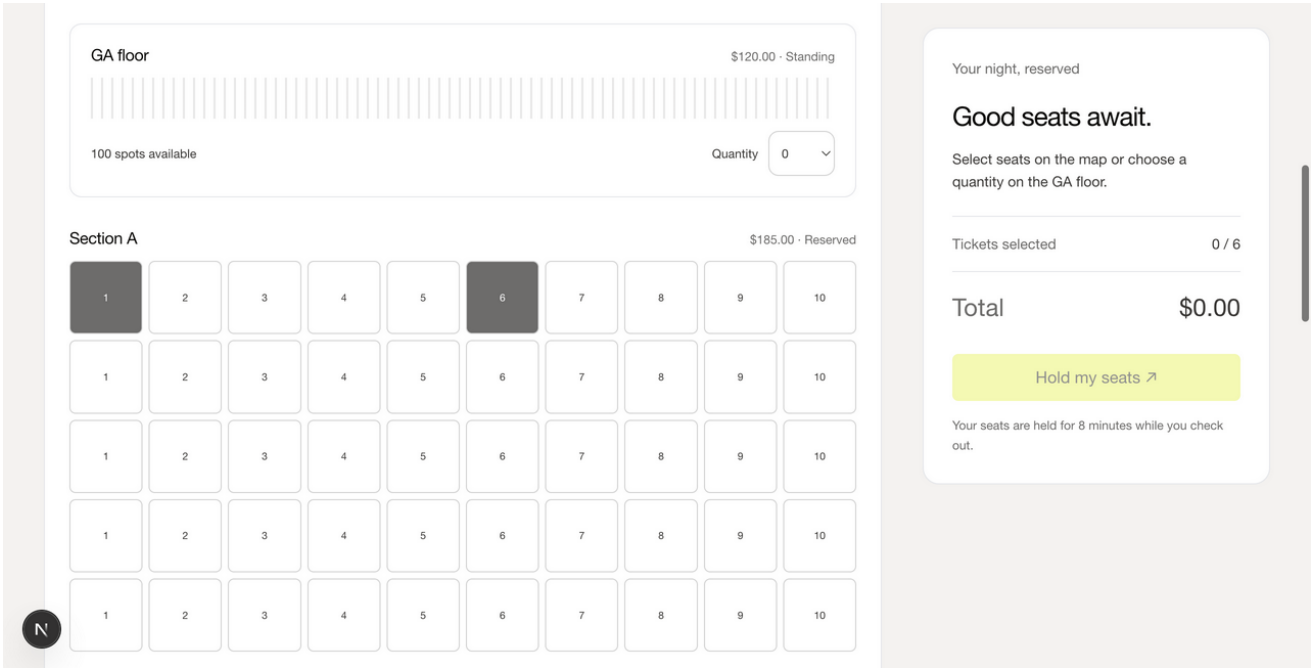


Figure 2. Reserved-seat selection and general admission at /events/1 (scrolled view).

Each show contains 50 premium, 120 standard, and 100 general-admission places. A customer may hold or buy up to six seats per show. Holds last eight minutes and restore after a reload. Approved checkout produces tickets; declined payment leaves the hold available for another attempt.

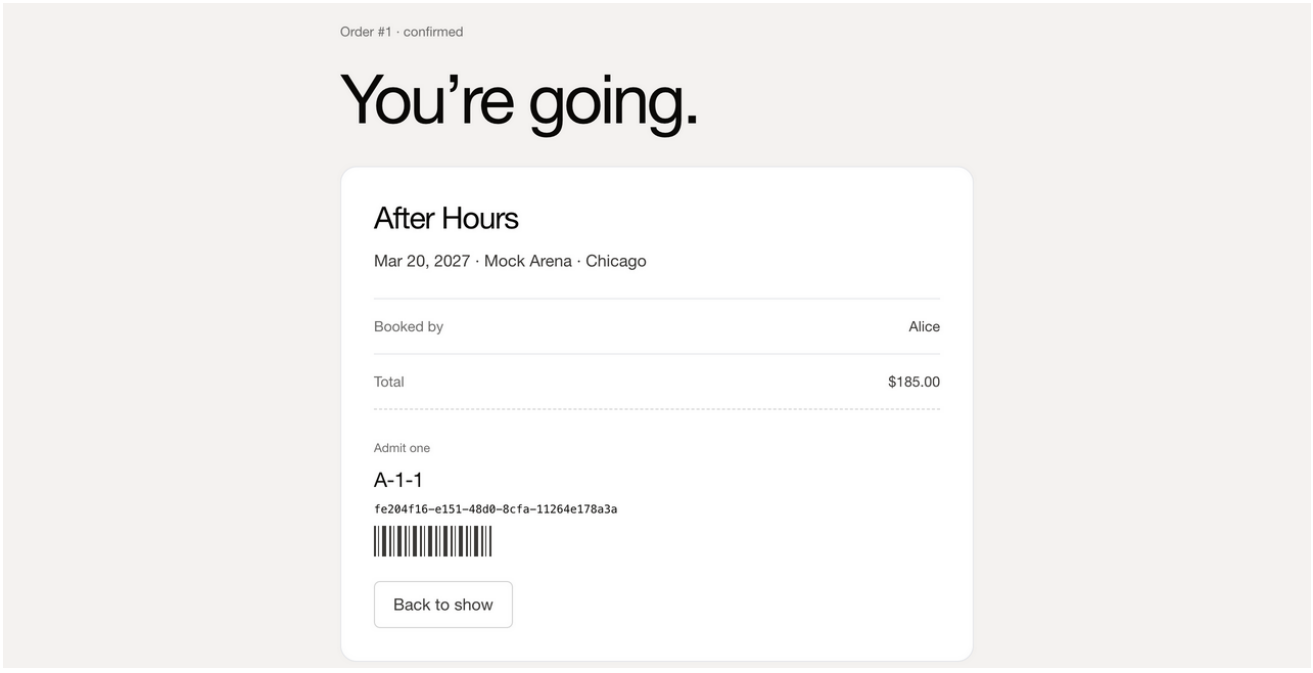


Figure 3. A confirmed synthetic purchase in /tickets/1. Screenshots show demonstration data; the ZIP seeds a fresh database.

Registration, login/logout, and profile editing are available. Reviews persist on event pages. The administrator can create, edit, publish, or hide shows. The interface supports light/dark themes, responsive layouts, and keyboard seat navigation.

Vulnerability demonstrations

V01 / SQL injection / A03 Injection

Location: backend/ticketsenpai/__init__.py, events(). Search interpolates user input into SQL LIKE expressions. In /events, search for the payload below; the unpublished Unannounced show appears alongside the three public shows. Its normal detail route still returns 404.

```
') OR 1=1 --
```

Impact: unauthorized event disclosure through query manipulation. Remediation: bind both LIKE parameters and preserve the publication predicate.

V02 / Stored XSS / A03 Injection

Location: review() and frontend/app/events/[id]/page.js. HTML is stored unchanged and rendered with dangerouslySetInnerHTML. Sign in as Alice, post this review, then sign in as Bob and fully reload /events/1:

```
<img src=x onerror="document.body.dataset.xss='verified'">
```

Evidence: evaluating document.body.dataset.xss returns verified under Bob's session. This harmless marker demonstrates persistent script execution. Impact: attacker-controlled JavaScript runs in another user's application context. Remediation: render plain text or apply an explicit HTML sanitization policy.

V03 / IDOR / A01 Broken Access Control

Location: order() at GET /api/orders/<id>. As Alice, reserve and buy a seat. Record the resulting order ID. Sign in as Bob: My Tickets omits Alice's purchase, but visiting /tickets/<Alice order ID> reveals its customer name, seat, and ticket identifier.

```
GET /api/orders/<Alice order ID>
Cookie: session=session-2
```

Impact: cross-account order disclosure. An active Bob session and an existing Alice order are prerequisites. Remediation: query by both order ID and authenticated user ID and return 404 when ownership differs.

Taxonomy: OWASP Top 10 (2021). SQL injection and XSS both map to A03; six weaknesses provide five primary categories. Full repeatable walkthroughs and source links are in docs/vulnerabilities.md.

Authentication and disclosure

V04 / Predictable sessions / A07

Location: login() and auth(). Login uses the deterministic token session-<user ID>, and there is no login lockout. First log in as Bob (bob@example.test / bob-demo); then use another client without logging in:

```
curl -s http://127.0.0.1:3000/api/me \
-H 'Cookie: session=session-2'
```

Evidence: the response identifies Bob. The target session must be active; sessions expire after 24 hours and logout revokes them. Impact: impersonation by token prediction. Remediation: generate random session tokens, rotate on login, retain expiry/revocation, and throttle login attempts.

V05 / Plaintext passwords / A02

Location: register(), login(), and backend/ticketsenpai/db.py. After make setup, inspect the local backend/instance/ticketsenpai.sqlite3 database:

```
SELECT email, password FROM users ORDER BY id;
```

Evidence: alice@example.test has readable password alice-demo; the other demo passwords are also readable. This standalone demonstration requires local database access, not an unauthenticated password endpoint. Impact: database disclosure reveals reusable credentials. Remediation: salted adaptive password hashing with migration or password resets.

V06 / Traceback disclosure / A05

Location: unexpected(). Malformed search syntax produces a JSON traceback:

```
curl --get --data-urlencode "q='broken" \
http://127.0.0.1:3000/api/events
```

Evidence: HTTP 500 with error.code = debug_error and error.traceback containing source paths and sqlite3.OperationalError. A lone apostrophe is not a reliable trigger. Impact: internal implementation details are exposed. Remediation: log exceptions server-side and return only an opaque error reference. The interactive Flask debugger is disabled.

Security review: these weaknesses are deliberate. A second teammate should review authentication/session behavior and reproduce each proof before peer testing. Do not substitute real credentials or payment data.

Delivery and team plan

Run and verify

Prerequisites: Node.js 22+ with npm, Python 3.10+ with venv, and make. Supported workflow: macOS, Linux, or Windows via WSL. Dependency installation needs internet access; the running application does not.

```
make setup
make dev
# Open http://127.0.0.1:3000; Ctrl+C stops both servers.
make check
make smoke # stop servers first; uses a temporary copy
```

Demo logins: alice@example.test / alice-demo; bob@example.test / bob-demo; admin@example.test / admin-demo. Only the seeded administrator can manage events. make reset CONFIRM=reset reseeds local data after servers stop.

Verification and development notes

make check passed 20 Flask tests and the Next.js production build. Flask coverage includes concurrency, expiry, idempotency, persistence, ownership boundaries, checkout failures, and the six vulnerable backend behaviors. Browser review separately verified stored XSS execution across users. Screenshots were captured from the running app.

No real payment gateway is used; checkout is simulated. The source ZIP omits local databases, installed dependencies, build output, and active session data.

Team

Bibas Kandel (Lead Developer), Bipul Adhikari, Om Nepal, Sushant Aryal, and Suwan Aryal. Bibas Kandel led development, the setup workflow, and packaging. The database, frontend, and security-testing work was shared across the team rather than assigned to one person.

Course-material alignment and remaining review

Reviewed the supplied Capstone Starter Template: Next.js + Flask + SQLite and separate frontend/backend directories align. TicketSenpai uses a package-based Flask API, port 5001, and a same-origin proxy instead of the starter's port 5000/CORS setup. The starter lists CSRF among examples; it does not establish a mandatory seventh weakness.